MODELS OF COMPUTATION

Project Report

Rule-Based Phishing URL Detector Using Finite Automata

Regular expressions → NFA → DFA → minimal DFA

Submitted by

Team Member	Register Number
Nilay Sengupta	25BYB0255
Adwit Agarwal	25BYB0116

Subject: Models of Computation (MOC)

Project Type: Mini Project

Faculty: Dr. Dharmendra Singh Rajput

October 2026

Abstract

Phishing attacks lure users to fraudulent websites through deceptive URLs, and many of these URLs share recognisable textual patterns. This project investigates how such patterns can be detected with the models studied in the Models of Computation course. The objective is an explainable, rule-based phishing URL detector in which every suspicious pattern is a regular language recognised by a finite automaton. A URL is modelled as a string over a 60-symbol alphabet Σ. Thirteen patterns, nine suspicious keywords, the "@" symbol, an IPv4 address used as the hostname, punycode hosts and "//" redirections, are written as regular expressions. Each expression is converted into an NFA with ε-moves, turned into a DFA by the subset construction, and reduced to a minimal DFA by partition refinement. The minimal DFAs read the URL, or its host or path component, one symbol at a time. Their accept or reject verdicts are combined with three structural rules into a weighted risk score, classified as Low Risk, Suspicious or High Risk. The engine was implemented in Python with a Flask API and an interactive web interface. Every minimal DFA agreed with Python's regular-expression engine on all 4.07 million test strings, scanned input in linear time (about 0.38 µs per character), and minimisation roughly halved each keyword automaton. On a 22-URL illustrative set, the detector classified 19 URLs as labelled. The project shows that regular languages and finite automata give a fast, verifiable and transparent basis for pattern-based URL screening, and makes their limits visible.

Keywords: Models of Computation, Finite Automata (DFA, NFA), Regular Expressions, Phishing URL Detection, Cybersecurity, Subset Construction, DFA Minimisation, Rule-Based Risk Scoring.

Contents

Abstract and Keywords

1. Introduction

1.1 Background

1.2 Problem Statement

1.3 Objectives

1.4 Scope of the Project

2. Theoretical Background

2.1 Fundamentals of the Selected Model

2.2 Formal Definition and Mathematical Representation

2.3 Relevant Theorems and Properties

2.4 Relevance to the Selected Application

3. Proposed Model and Methodology

3.1 Problem Formulation

3.2 System Architecture and Workflow

3.3 Design of the Computational Model

4. Implementation and Results

4.1 Tools and Technologies Used

4.2 Experimental Results

5. Applications, Discussion and Limitations

5.1 Practical Applications

5.2 Limitations and Challenges

5.3 Discussion

6. Conclusion and Future Scope

6.1 Conclusion

6.2 Future Enhancements

References

Appendix A: Project Files and How to Run

1. Introduction

1.1 Background

Phishing is a social-engineering attack in which an attacker imitates a trusted service, such as a bank, an e-mail provider or a university portal, to trick a user into revealing credentials or installing malware. The link the victim is asked to open is the attacker's main delivery mechanism, so the URL itself often carries the first warning signs: words such as *login*, *verify* or *account* placed in unusual positions, an "@" that hides the real host, a raw IP address instead of a domain name, unusually many subdomain levels, or encoded internationalised hostnames.

Production systems detect phishing with blocklists, reputation services and machine-learning classifiers. Those methods are effective but often opaque. They need large labelled datasets, and their decisions are hard to explain. A Models of Computation course offers another view of the same problem: a URL is a finite string over a finite alphabet, and a "suspicious pattern" is a set of strings, that is, a formal language. When that language is regular, a finite automaton can recognise it exactly, in a single left-to-right pass, with constant memory.

This project builds a phishing URL detector on that idea. It does not claim that finite automata solve phishing detection. Instead it shows, with working code and measured results, how regular patterns that occur in URLs can be specified formally, compiled into automata by the standard constructions, verified, and combined into a transparent risk assessment.

1.2 Problem Statement

Design and implement a rule-based system that accepts a URL as a string, analyses it for predefined suspicious patterns using finite automata, performs additional structural checks, and produces an interpretable risk classification. Every pattern must be expressed as a regular expression and recognised by a deterministic finite automaton that is derived from the expression by the formal constructions of automata theory (regular expression → NFA → DFA → minimal DFA), rather than coded by hand.

1.3 Objectives

- Model URLs as strings over a defined alphabet Σ of URL characters.
- Express suspicious URL patterns as regular expressions, and hence as regular languages.
- Convert each regular expression into an NFA with ε-moves, convert the NFA into an equivalent DFA by the subset construction, and minimise the DFA.
- Implement a DFA engine with explicit states, transition function, start state and accepting states that processes a URL one character at a time.
- Verify that every compiled automaton accepts exactly the language of its regular expression.

- Combine the automata verdicts with structural URL rules into an interpretable risk score and a final class.
- Present the system through a web interface that also visualises the automata and their step-by-step computation.

1.4 Scope of the Project

The project covers lexical analysis of the URL string only. It parses the URL into components, recognises 13 patterns with minimal DFAs, applies three structural rules, and assigns one of three risk classes. It does not fetch the web page, query DNS or WHOIS, use blocklists, or apply machine learning. Patterns are kept simple and readable for teaching purposes, and the weights and thresholds are project-defined. The risk score is therefore an educational heuristic, not a security verdict. The evaluation uses a small illustrative URL set written by the team, so it demonstrates behaviour rather than estimating real-world accuracy.

2. Theoretical Background

2.1 Fundamentals of the Selected Model

The selected models of computation are **regular expressions**, **nondeterministic finite automata (NFA)** and **deterministic finite automata (DFA)**, the three equivalent descriptions of the class of regular languages.

Alphabet, strings and languages. An alphabet Σ is a finite, non-empty set of symbols. A string over Σ is a finite sequence of symbols; ε denotes the empty string and Σ^* the set of all strings. A language is any subset $L \subseteq \Sigma^*$. In this project Σ contains the 59 characters permitted in a URL by RFC 3986 after lower-casing (a–z, 0–9 and - . _ ~ : / ? # [ ] @ ! $ & ' ( ) * + , ; = %) together with one extra symbol $\perp$ that stands for any other character, giving $|\Sigma| = 60$.

Regular expressions describe languages using three operations: union ($r_1 \mid r_2$), concatenation ($r_1 r_2$) and Kleene star (r^*). The detector also uses the standard shorthands $r+ = rr^*$, $r? = r \mid \varepsilon$, character classes such as [0-9], and "." for any symbol of Σ.

Finite automata are abstract machines with a finite set of states and no other memory. A DFA has exactly one next state for each state and input symbol. An NFA may have zero, one or several next states, and may also change state on ε without reading input. NFAs are convenient to construct from regular expressions, while DFAs are convenient to execute.

2.2 Formal Definition and Mathematical Representation

A **deterministic finite automaton** is a 5-tuple

$$M = (Q, \Sigma, \delta, q_0, F)$$

where Q is a finite set of states, Σ is the input alphabet, $\delta : Q \times \Sigma \to Q$ is the transition function, $q_0 \in Q$ is the start state and $F \subseteq Q$ is the set of accepting states. The extended transition function $\hat{\delta} : Q \times \Sigma^* \to Q$ applies δ along a string:

$$\hat{\delta}(q, \varepsilon) = q, \qquad \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$$

and the language accepted by M is

$$L(M) = \{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \}.$$

A **nondeterministic finite automaton with ε-moves** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$ in which the transition function returns a set of states, $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$. The ε-closure of a set S, written E(S), is the set of states reachable from S by ε-moves alone. N accepts w if at least one sequence of choices ends in a state of F.

Example: the keyword "login". The language of URLs that contain the keyword is

$$L_login = \Sigma^* \cdot login \cdot \Sigma^*, \qquad \text{written as the regular expression } .*login.*$$

Its minimal DFA (derived in Section 3.3) has $Q = \{q_0, \ldots, q_5\}$, start state q_0 and $F = \{q_5\}$. State q_i means that the longest suffix of the input read so far which is also a prefix of "login" has length i. For example, q_2 means that the input so far ends with "lo". Once q_5 is reached, the

keyword has occurred and q_5 loops on every symbol of Σ, so the URL is accepted whatever follows.

Symbol classes. To keep transition tables small, each pattern partitions Σ into the coarsest set of classes whose members the expression cannot tell apart. For .*login.* the classes are {g}, {i}, {l}, {n}, {o} and "other" (the remaining 55 symbols), so δ needs 6 columns rather than 60. This is the character-class technique used by lexer generators; it changes the table size, not the language recognised.

2.3 Relevant Theorems and Properties

Theorem 1 (Kleene). A language is denoted by a regular expression if and only if it is accepted by some finite automaton. The constructive direction, regular expression → NFA, builds an NFA with ε-moves from two-state fragments: one fragment per symbol, joined by ε-moves for concatenation, union and star. The resulting NFA has at most 2|r| states, one start state and one accepting state.

Theorem 2 (Rabin–Scott subset construction). For every NFA N there is a DFA D with L(D) = L(N). The states of D are sets of states of N:

$$q_D0 = E(\{q_0\}), \qquad \delta_D(S, a) = E(\ \bigcup_\{q \in S\}\ \delta_N(q, a)\ ), \qquad F_D = \{\ S \mid S \cap F_N \neq \varnothing\ \}$$

In the worst case D has 2^|Q_N| states. Only subsets reachable from q_D0 are generated in practice, and the empty set becomes the dead (trap) state.

Theorem 3 (Myhill–Nerode). For a language L, define $x \equiv_L y$ if, for every $z \in \Sigma^*$, $xz \in L \Leftrightarrow yz \in L$. L is regular if and only if $\equiv_L$ has finitely many equivalence classes, and the minimal DFA for L has exactly one state per class. It is unique up to renaming of states. Moore's partition-refinement algorithm computes it: start from the partition {F, Q\F} and repeatedly split any block whose states move to different blocks on some symbol, until no block splits.

Theorem 4 (closure properties). Regular languages are closed under union, intersection, complement, concatenation and Kleene star. Hence "contains login *or* verify" is regular, and the nine keyword DFAs can be merged into one automaton (Section 4.2 measures this).

Property 5 (linear-time recognition). A DFA reads each input symbol once and does one table lookup per symbol, so deciding $w \in L(M)$ takes O(|w|) time and O(1) extra memory, independent of how complicated the pattern was.

Theorem 6 (pumping lemma). If L is regular there is a constant p such that every $w \in L$ with $|w| \geq p$ can be written w = xyz with $|xy| \leq p$, $|y| \geq 1$ and $xy^iz \in L$ for all $i \geq 0$. It proves that some languages, such as $\{a^nb^n \mid n \geq 0\}$ or balanced brackets, are not regular. This marks the boundary of what the automata part of the detector can do, which is why counting rules are implemented outside the automata (Section 3.2).

2.4 Relevance to the Selected Application

The suspicious URL patterns targeted by the detector are naturally regular. “Contains a keyword” is Σ*wΣ*, “contains @” is Σ*@Σ*, and “is a dotted-quad of digit strings” is [0-9]+(\.[0-9]+)³. Kleene's theorem guarantees that each pattern has a DFA. The subset construction and minimisation produce that DFA mechanically and optimally, and Property 5 makes the scan fast. Each verdict can also be explained by a concrete computation: the sequence of states visited and the accepting state reached. The pumping lemma explains what must stay outside the automata, such as unbounded counting or comparing two parts of the URL. Table 1 maps each MOC concept to its role in the project.

Table 1. Mapping of MOC concepts to the project

MOC concept	**Symbol**	**Application in the project**
Alphabet	Σ	59 URL characters plus ⊥, compressed into symbol classes per pattern
String	w ∈ Σ*	The normalised URL, or its host or path component
Language	L ⊆ Σ*	The set of URLs that contain a given suspicious pattern
Regular expression	r	.*login.*, .*@.*, [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+, …
Regular language	L(r)	The language recognised by each of the 13 automata
NFA with ε-moves	N	First automaton built from each regular expression
Subset construction	$\mathscr{P}$(Q_N)	Converts every NFA into an equivalent complete DFA
DFA	(Q, Σ, δ, q_0, F)	The deterministic recogniser that scans the URL
States	Q	How much of the pattern has been matched so far
Transitions	δ	Table lookup δ[q][class(c)], including fall-back moves after a partial match
Start state	q_0	Nothing of the pattern matched yet
Accepting state	F	Pattern recognised; for Σ*wΣ* it loops on all of Σ
Dead / reject state	q_d	Pattern can no longer be completed (IPv4 DFA after a letter)
Minimisation	≡_L	Moore's refinement yields the smallest DFA for each pattern

3. Proposed Model and Methodology

3.1 Problem Formulation

Let u be the URL entered by the user. Normalisation lower-cases u and maps every character outside the URL alphabet to ⊥, giving a string $w \in \Sigma^*$. A parser splits w into components: scheme, userinfo, host, port, path, query and fragment. The host is further split into subdomain and registered domain.

The detector holds a catalogue of patterns $P_1, \ldots, P_{13}$. Each pattern $P_i = (r_i, c_i, w_i)$ consists of a regular expression r_i, the URL component $c_i \in \{\text{url, host, path}\}$ that it reads, and a weight w_i. For each pattern the system computes the minimal DFA M_i with $L(M_i) = L(r_i)$ and the Boolean verdict

$$d_i = [\ \hat{\delta}_i(q_0, \text{component_}c_i(w)) \in F_i\ ]$$

Three structural predicates s_1, s_2, s_3 (URL length, number of host labels, number of special characters) are evaluated directly. The risk score and class are

$$\text{score}(u) = \Sigma_i\, w_i \cdot d_i + \Sigma_j\, s_j, \qquad \text{class} = \text{Low Risk if score} \le 1;\ \text{Suspicious if } 2 \le \text{score} \le 3;\ \text{High Risk if score} \ge 4$$

The weights and thresholds are design choices of the project, not mathematical properties of finite automata. The detector is deliberately a *combination* of several regular languages and ordinary rules. It is not claimed that "phishing URL" is itself a regular language.

Table 2. Pattern catalogue and risk-scoring weights

Pattern	Regular expression	Reads	Weight
Keywords (9): login, signin, verify, account, password, secure, update, bank, confirm	.*k.* for each keyword k	url	+1 each
"@" symbol in URL	.*@.*	url	+2
IPv4 address as hostname	[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+	host	+2
Punycode host	.*xn--.*	host	+1
"//" redirect inside path	.*//.*	path	+1
URL longer than 75 characters	structural rule	url	+1
More than 3 hostname labels (not counted for IP hosts)	structural rule	host	+1
More than 6 special characters (- _ = & % ~)	structural rule	url	+1

3.2 System Architecture and Workflow

The system is divided into four modules (Figure 1). The automata are responsible only for pattern recognition. Parsing, the counting rules and scoring live in separate modules, which keeps each part simple and keeps the formal claims precise.



Figure 1. The four modules of the detector

1. **URL parser** (`parse_url`) normalises case and extracts the components. The real host is taken after the *last* "@" in the authority part, which exposes the classic trick `http://mybank.com@203.0.113.7/`, where the browser actually connects to 203.0.113.7.
2. **MOC automata engine** (`compile_regex`, `DFA.run`) parses each regular expression, builds the NFA, applies the subset construction and minimisation, and runs the minimal DFA on the chosen component.
3. **Suspicious-feature detector** (`analyse`, `classify`) collects the 13 DFA verdicts, evaluates the 3 structural rules and computes the score and class.
4. **Web interface** (`app.py`, `website/index.html`) shows the analysis, the extracted components and each DFA's verdict. It lets the user step through any DFA's computation and build automata from any regular expression.

The workflow is: User URL → preprocessing → component extraction → 13 DFAs and 3 rules → pattern results → risk score → Low / Suspicious / High.

Counting conditions such as "more than 3 labels" could in principle be written as regular languages, because the bounds are fixed. They are implemented as plain rules because that is simpler and clearer. Conditions with no fixed bound, such as comparing the displayed link text with the real host, are not regular at all (Theorem 6).

3.3 Design of the Computational Model

Every pattern passes through the same four-stage compilation pipeline. The stages are illustrated here for the keyword pattern `.*login.*`.

Stage 1: Alphabet compression

The parser collects the character sets used by the expression ({l}, {o}, {g}, {i}, {n} and Σ for each "."). The coarsest partition of Σ that respects all of them gives the symbol classes g, i, l, n, o and other. All later stages work over these 6 classes.

Stage 2: Regular expression → NFA

Each symbol becomes a two-state fragment s —a→ f. Concatenation links fragments with ε-moves, and a star adds ε-moves that allow skipping or repeating its fragment. For .*login.* this gives an NFA with 18 states and 14 ε-moves, with start state 0 and accepting state 15 (Figure 2). States 0–4 implement the leading Σ*, states 5–13 spell l-o-g-i-n, and states 14–17 implement the trailing Σ*.

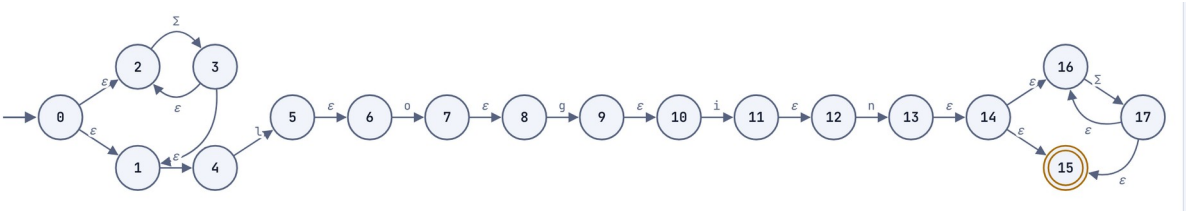


Figure 2. NFA with ε-moves for .*login.* (start state 0, accepting state 15)

Stage 3: Subset construction (NFA → DFA)

Starting from E({0}) = {0, 1, 2, 4}, the algorithm computes E(move(S, a)) for every reachable set S and every symbol class a. Table 3 lists all 13 reachable subsets. A subset is accepting when it contains NFA state 15. Although 2^{18} = 262,144 subsets are possible, only 13 are reachable.

Table 3. Subset construction for .*login.* (→ start, * accepting)

D-state	ε-closure (NFA states)	g	i	l	n	o	other
→D0	{0, 1, 2, 4}	D1	D1	D2	D1	D1	D1
D1	{1, 2, 3, 4}	D1	D1	D2	D1	D1	D1
D2	{1, 2, 3, 4, 5, 6}	D1	D1	D2	D1	D3	D1
D3	{1, 2, 3, 4, 7, 8}	D4	D1	D2	D1	D1	D1
D4	{1, 2, 3, 4, 9, 10}	D1	D5	D2	D1	D1	D1
D5	{1, 2, 3, 4, 11, 12}	D1	D1	D2	D6	D1	D1
*D6	{1, 2, 3, 4, 13, 14, 15, 16}	D7	D7	D8	D7	D7	D7
*D7	{1, 2, 3, 4, 15, 16, 17}	D7	D7	D8	D7	D7	D7
*D8	{1, 2, 3, 4, 5, 6, 15, 16, 17}	D7	D7	D8	D7	D9	D7
*D9	{1, 2, 3, 4, 7, 8, 15, 16, 17}	D10	D7	D8	D7	D7	D7
*D10	{1, 2, 3, 4, 9, 10, 15, 16, 17}	D7	D11	D8	D7	D7	D7
*D11	{1, 2, 3, 4, 11, 12, 15, 16, 17}	D7	D7	D8	D12	D7	D7
*D12	{1, 2, 3, 4, 13, 14, 15, 16, 17}	D7	D7	D8	D7	D7	D7

Stage 4: Minimisation (Moore's partition refinement)

The initial partition separates accepting from non-accepting D-states. Every round splits off one more non-accepting state, according to how far it is from completing the keyword. After 4 refining rounds the partition is stable. All seven accepting D-states (D6–D12) are equivalent, because once “login” has been seen every continuation is accepted. The result has 6 states (Figure 3).

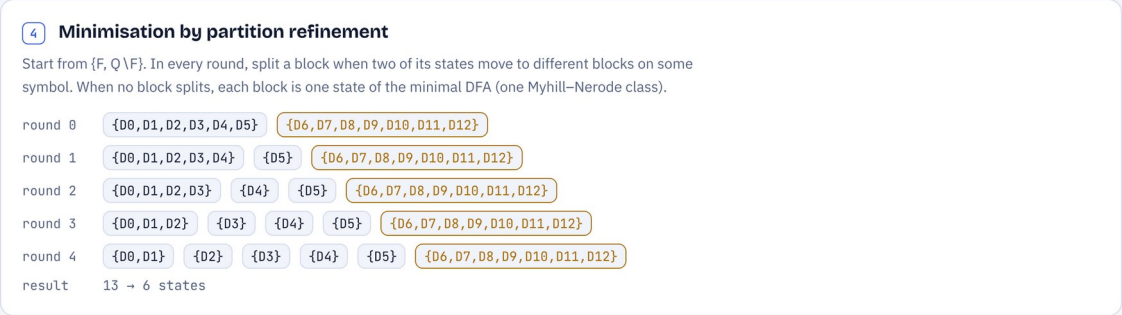


Figure 3. Partition-refinement rounds for the .*login.* DFA (accepting blocks outlined)

States are renamed q_0, q_1, … in breadth-first order from the start state. The resulting minimal DFA (Figure 4, Table 4) is the $q_0 \to q_1 \to \dots \to q_5$ chain from the project proposal, completed with the fall-back transitions needed for substring search. For example, $\delta(q_2, l) = q_1$: after reading "lol", the last "l" may begin a new occurrence of "login", so the machine falls back to q_1 rather than to q_0. These are exactly the failure transitions of the Knuth–Morris–Pratt algorithm, obtained here automatically by the subset construction.

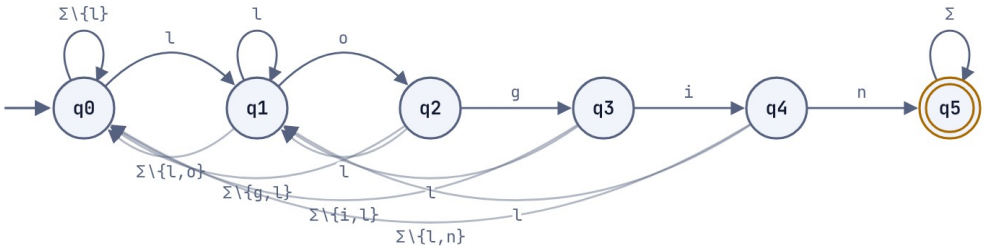


Figure 4. Minimal DFA for .*login.*; F = {q5}. Edges back to q0 or q1 are the fall-back transitions

Table 4. Transition table of the minimal .*login.* DFA (→ start, * accepting)

δ	g	i	l	n	o	other
→q0	q0	q0	q1	q0	q0	q0
q1	q0	q0	q1	q0	q2	q0
q2	q3	q0	q1	q0	q0	q0
q3	q0	q4	q1	q0	q0	q0
q4	q0	q0	q1	q5	q0	q0
*q5	q5	q5	q5	q5	q5	q5

A pattern with a dead state: IPv4 hostname

The IPv4 pattern must match the *whole* host, so it is not wrapped in Σ*…Σ*. Its NFA has 22 states. The subset construction gives 9 DFA states, one of which is the empty set ∅. Minimisation leaves all 9 distinct, including the dead state q_d (Figure 5, Table 5). Any letter, or a misplaced dot, sends the machine to q_d, where it stays.

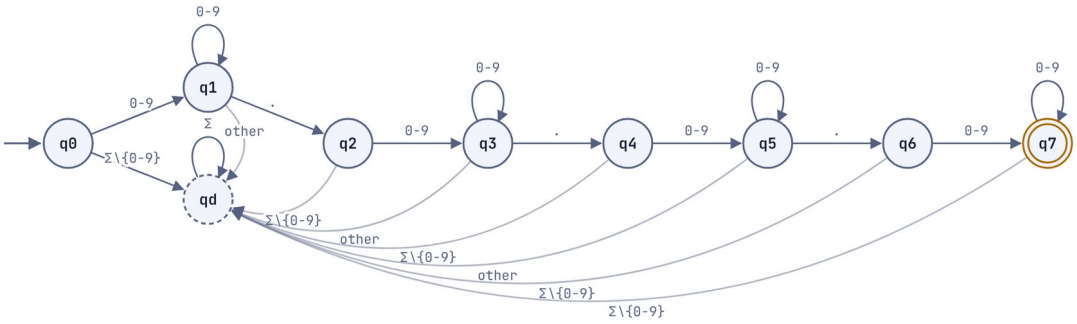


Figure 5. Minimal DFA for [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+ (dashed: dead state q_d)

Table 5. Transition table of the IPv4-host DFA

δ	**0-9**	**.**	**other**
→q0	q1	qd	qd
q1	q1	q2	qd
q2	q3	qd	qd
q3	q3	q4	qd
q4	q5	qd	qd
q5	q5	q6	qd
q6	q7	qd	qd
*q7	q7	qd	qd
qd	qd	qd	qd

DFA engine

The engine stores the 5-tuple directly. A run is the extended transition function $\hat{\delta}$, and it returns the whole state trace so that the interface can animate it:

```
@dataclass
class DFA:
    states: list[str]            # Q
    classes: list[frozenset]     # Σ, compressed into symbol classes
    delta: list[list[int]]       # δ[q][class] -> q'
    start: int                   # q0
    accepting: set[int]          # F

    def run(self, text):         # extended transition function δˆ
        q, trace = self.start, [self.start]
        for ch in text:
            q = self.delta[q][self.symbol_of(ch)]
            trace.append(q)
        return trace

    def accepts(self, text):
        return self.run(text)[-1] in self.accepting
```

Worked example. For `http://login.example.com/verify-account`, the login DFA moves $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \rightarrow q_5$ while reading “login” and then stays in q_5, so it accepts. The verify and account DFAs also accept. No other DFA or rule fires, so the score is 1 + 1 + 1 = 3 and the URL is classed **Suspicious**, the same result predicted in the project proposal.

4. Implementation and Results

4.1 Tools and Technologies Used

Table 6. Tools and technologies

Component	Technology	Purpose
Automata engine	Python 3 (standard library only)	Regex parser, NFA construction, subset construction, minimisation, DFA execution
Detector	Python 3	URL parsing, pattern catalogue, structural rules, scoring
Back end	Flask	/api/analyze, /api/automaton/<id> and /api/compile JSON endpoints
Front end	HTML, CSS, JavaScript (no frameworks)	Analyzer, step-by-step DFA simulator, construction lab, results
Visualisation	Inline SVG generated in JavaScript	State diagrams, tape animation, gauges and charts
Testing	pytest, Python re as reference oracle	Unit tests and DFA-versus-regex equivalence checks
Evaluation	evaluate.py	Reproduces every number reported in this chapter

The browser version of the engine is a line-for-line JavaScript port of the Python engine, so the website works without a server. A parity check confirmed that both produce identical automata (all state counts) and identical scores and indicators for every URL in the evaluation set.

4.2 Experimental Results

Experiment 1: Automaton sizes through the pipeline

Table 7 reports the size of every automaton at each stage, and Figure 6 plots it.

Table 7. Number of states at each stage of the compilation pipeline

Pattern	Σ classes	NFA states	ε-moves	Subset DFA	Minimal DFA
login	6	18	14	13	6
signin	5	20	15	15	7
verify	7	20	15	15	7
account	7	22	16	17	8
password	8	24	17	19	9
secure	6	20	15	15	7
update	7	20	15	15	7
bank	5	16	13	11	5
confirm	8	22	16	17	8
at_symbol	2	10	10	5	2
ipv4_host	3	22	18	9	9

Pattern	Σ classes	NFA states	ε-moves	Subset DFA	Minimal DFA
punycode	4	16	13	11	5
dbl_slash	2	12	11	7	3
union of 9 keywords	22	120	74	111	44

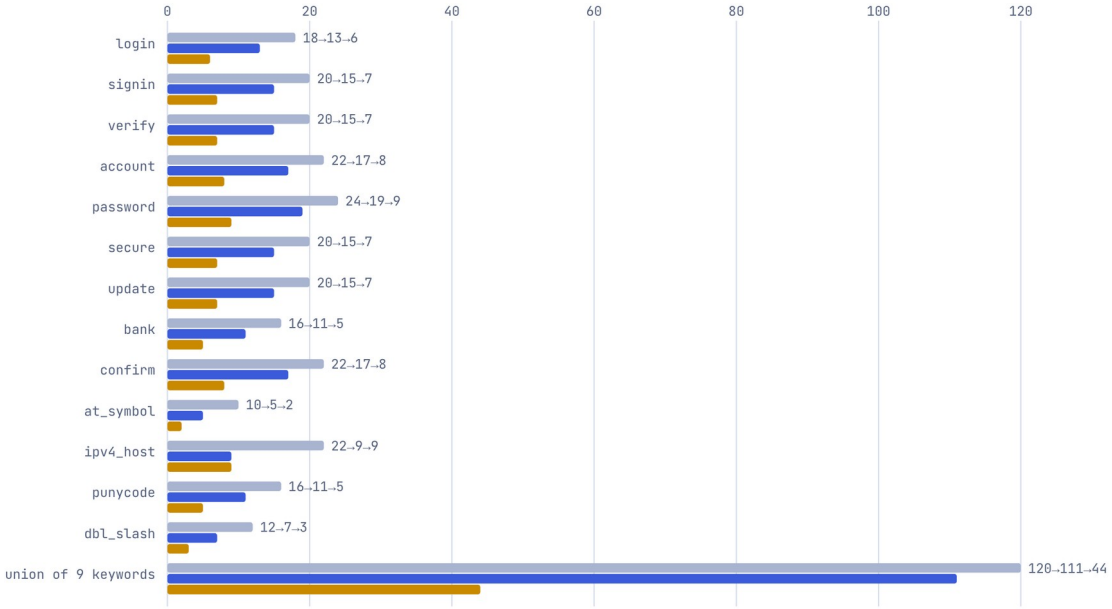


Figure 6. States per pattern: NFA, subset-construction DFA and minimal DFA

Observations. (i) The subset construction stayed far below its 2^n worst case. Every DFA had fewer states than the NFA it came from (for example 18 → 13 for login and 120 → 111 for the union). (ii) Minimisation roughly halved every keyword DFA. A keyword of length k always ends with exactly k + 1 states, which matches the Myhill–Nerode count: one class for each matched prefix length. (iii) The IPv4 DFA was already minimal after the subset construction (9 → 9). (iv) Merging the nine keywords into one regular expression gives a single 44-state DFA, compared with 64 states for the nine separate DFAs. The single DFA is smaller, but it only reports that *some* keyword occurred. The project keeps separate DFAs so that each keyword is reported and weighted individually.

Experiment 2: Correctness of the compiled automata

Each minimal DFA was compared with Python's `re.fullmatch` on the same inputs, using two tests: 20,000 random strings per pattern, biased towards the pattern's own characters so that matches are common, and an exhaustive test of every string up to length L built from one representative character per symbol class. In total 4,067,345 strings were checked, and the DFA and `re` agreed on 4,067,345 of them (100%).

Table 8. DFA versus Python re: agreement on identical inputs

Pattern	Random agreement	Exhaustive agreement	L
login	20,000 / 20,000	335,923 / 335,923	7

Pattern	Random agreement	Exhaustive agreement	L
signin	20,000 / 20,000	97,656 / 97,656	7
verify	20,000 / 20,000	137,257 / 137,257	6
account	20,000 / 20,000	137,257 / 137,257	6
password	20,000 / 20,000	299,593 / 299,593	6
secure	20,000 / 20,000	335,923 / 335,923	7
update	20,000 / 20,000	137,257 / 137,257	6
bank	20,000 / 20,000	97,656 / 97,656	7
confirm	20,000 / 20,000	299,593 / 299,593	6
at_symbol	20,000 / 20,000	524,287 / 524,287	18
ipv4_host	20,000 / 20,000	265,720 / 265,720	11
punycode	20,000 / 20,000	349,525 / 349,525	9
dbl_slash	20,000 / 20,000	524,287 / 524,287	18
union of 9 keywords	20,000 / 20,000	245,411 / 245,411	4

In addition, 17 unit tests pass. They include the textbook expression (a|b)*abb, whose minimal DFA has 4 live states as expected, and checks for ε in alternation, negated classes, and the fall-back transition needed to accept “lologin”.

Experiment 3: Detection on an illustrative URL set

The team wrote 22 URLs: 10 benign URLs of well-known sites and 12 phishing-style URLs that use invented or reserved domains (such as example.com and the 203.0.113.0/24 documentation range) to imitate common phishing structures. A URL counts as *flagged* when it is classed Suspicious or High Risk. Table 9 lists the results, and Figure 7 shows the analyzer for one of them.

Table 9. Detector output on the illustrative URL set

URL	Label	Score	Class	Indicators detected
`https://www.google.com/`	benign	0	Low Risk	—
`https://github.com/nilay-11`	benign	0	Low Risk	—
`https://en.wikipedia.org/wiki/Finite-state_machine`	benign	0	Low Risk	—
`https://vit.ac.in/academics`	benign	0	Low Risk	—
`https://docs.python.org/3/library/re.html`	benign	0	Low Risk	—
`https://www.amazon.in/gp/cart/view.html`	benign	0	Low Risk	—
`https://stackoverflow.com/questions/tagged/regex`	benign	0	Low Risk	—
`https://news.ycombinator.com/item?id=1`	benign	0	Low Risk	—

URL	Label	Score	Class	Indicators detected
`https://accounts.google.com/signin`	benign	2	Suspicious	signin, account
`https://www.icicibank.com/`	benign	1	Low Risk	bank
`http://login.example.com/verify-account`	phish	3	Suspicious	login, verify, account
`http://login-example.com/verify`	phish	2	Suspicious	login, verify
`http://192.168.10.5/secure/bank/login.php`	phish	5	High Risk	login, secure, bank, ipv4_host
`http://mybank.com@203.0.113.7/confirm`	phish	6	High Risk	bank, confirm, at_symbol, ipv4_host
`http://secure-update.mybank.verify.example.net/account/login`	phish	7	High Risk	login, verify, account, secure, update, bank, many_subdomains
`https://mybank.example-secure-login.xyz/password/reset?user=abc&token=xyz&step=2&ref=mail`	phish	6	High Risk	login, password, secure, bank, long_url, special_chars
`http://xn--mybnk-9qa.example/signin`	phish	2	Suspicious	signin, punycode
`http://198.51.100.23/~user/account_update.html`	phish	4	High Risk	account, update, ipv4_host
`https://example.org//redirect//login.mybank.example/`	phish	3	Suspicious	login, bank, dbl_slash
`http://update.mybank.com.account-verify.example.ru/confirm`	phish	6	High Risk	verify, account, update, bank, confirm, many_subdomains
`http://bit.ly/3xYzAbc`	phish	0	Low Risk	—
`https://mybank-support.example.top/help`	phish	1	Low Risk	bank

Table 10. Confusion matrix on the illustrative set

	Flagged	Not flagged
Phishing-style (12)	10 (true positive)	2 (false negative)
Benign (10)	1 (false positive)	9 (true negative)

The detector classified 19 of the 22 URLs as labelled. The three errors show its limits clearly. `accounts.google.com/signin` is legitimate but contains two keywords (score 2). `bit.ly/3xYzAbc` hides its destination behind a shortener and contains no pattern (score 0). `mybank-support.example.top/help` imitates a brand without any listed keyword (score 1). Because this set is small and was written by the team, these figures illustrate behaviour and are not an accuracy estimate.

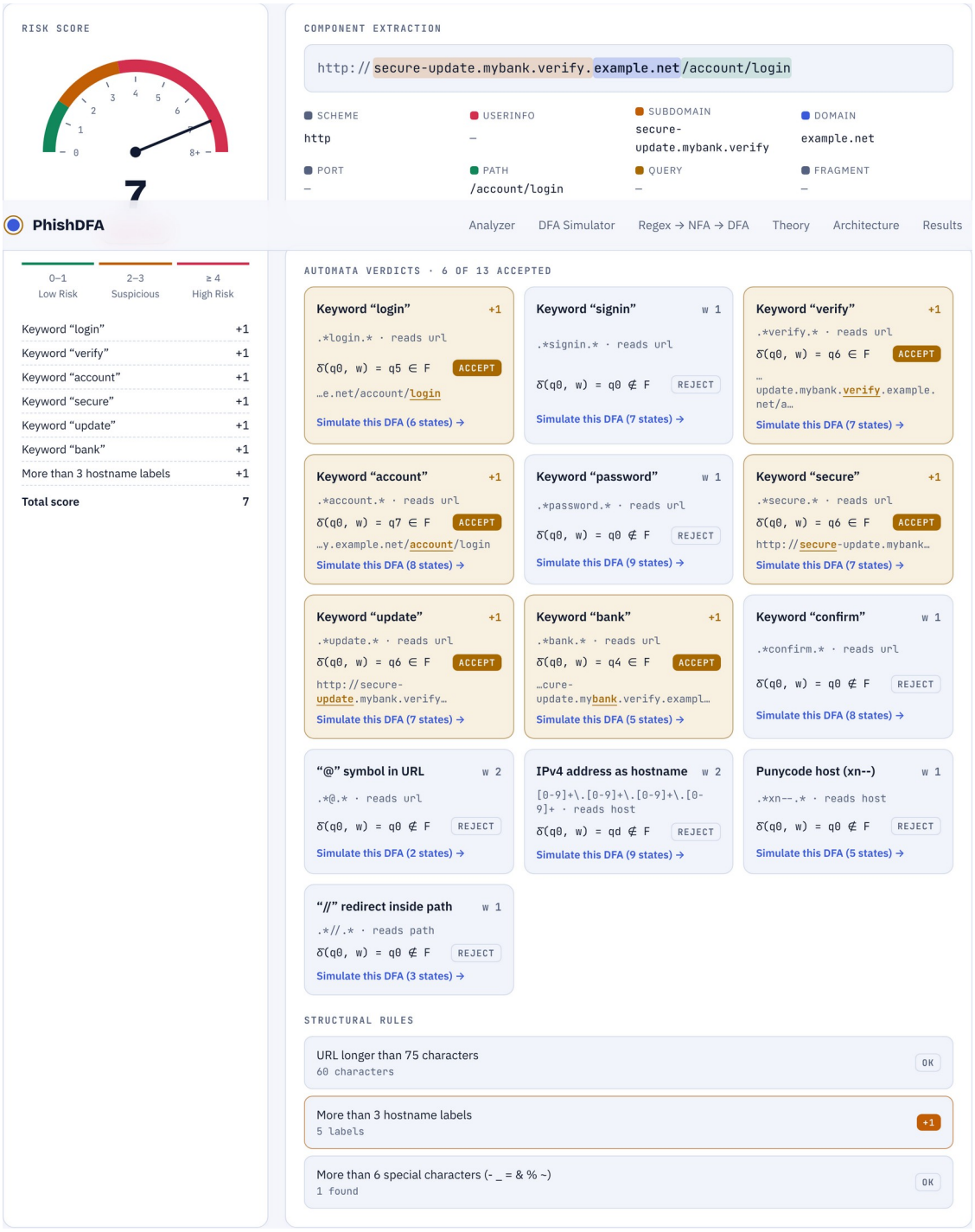


Figure 7. Web interface: risk gauge, component extraction and the 13 DFA verdicts for a phishing-style URL

Experiment 4: Running time

Because a DFA does one lookup per character, scan time should grow linearly with input length. Table 11 and Figure 8 confirm this. The time per character stays constant at about 0.389 µs from 1,000 to 16,000 characters. A complete analysis (parsing, 13 DFAs and 3 rules) took 0.323 ms per URL on average in pure Python. The JavaScript engine in the browser is considerably faster still.

Table 11. Scan time of the login DFA versus input length

Input length (characters)	DFA scan time (ms)	Time per character (µs)
1,000	0.389	0.389
2,000	0.763	0.382
4,000	1.573	0.393
8,000	3.165	0.396
16,000	6.088	0.38

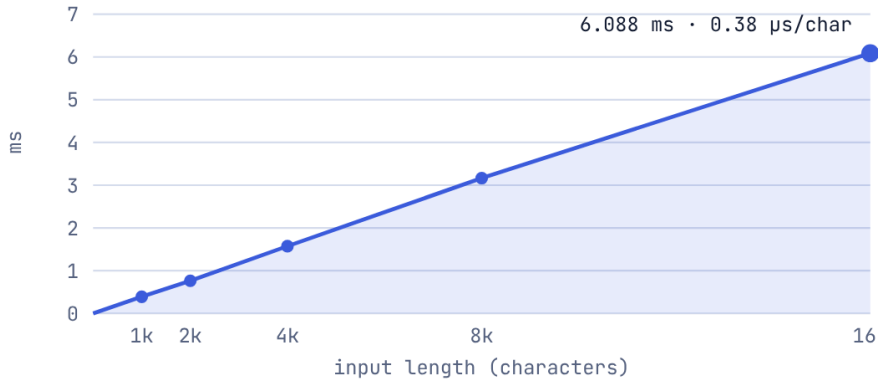


Figure 8. DFA scan time grows linearly with input length (O(n))

Interactive demonstration

The website includes a step-by-step simulator (Figure 9). The user picks a pattern and an input, then steps forward and backward through the computation. The current state is highlighted in the diagram, the tape shows the state after each symbol, and the transition table highlights the entry δ(q, a) being applied. At the end the simulator reports whether the final state is in F. A construction lab accepts any regular expression and shows all four stages: symbol classes, NFA diagram, subset-construction table, refinement rounds and the minimal DFA.

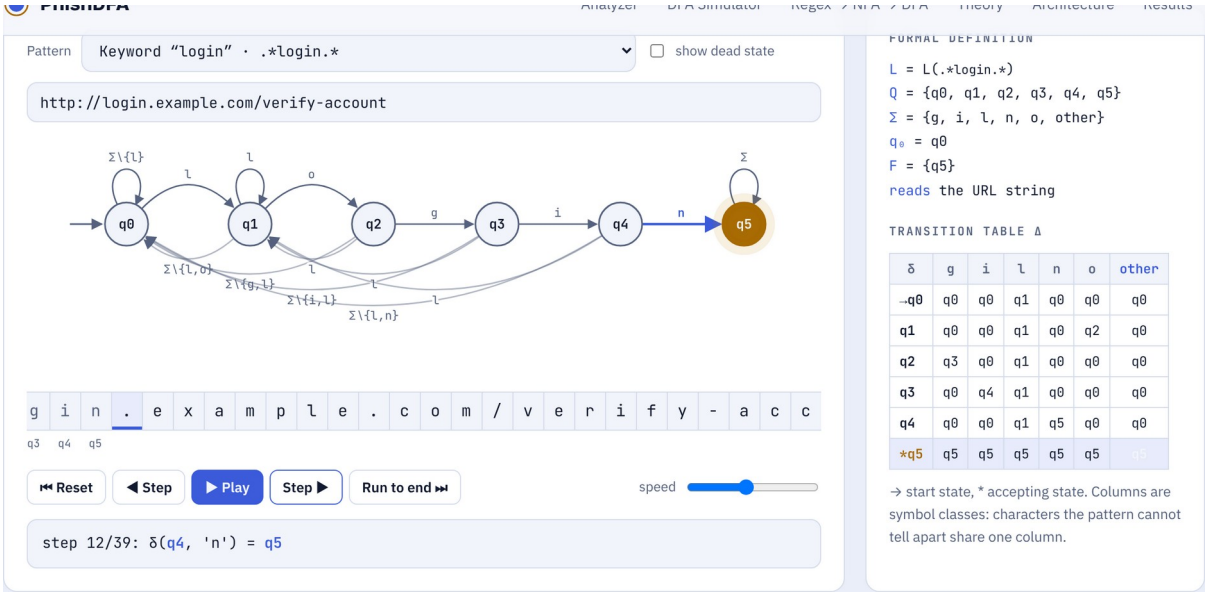


Figure 9. Step-by-step DFA simulator after 12 steps on http://login.example.com/verify-account

5. Applications, Discussion and Limitations

5.1 Practical Applications

- **First-line URL screening** in e-mail gateways, chat applications or browser extensions, where a fast, explainable pre-filter decides which links need deeper inspection.
- **User awareness tools** that explain *why* a link looks risky ("the real host is after the @", "IP address used as hostname") instead of giving an opaque verdict.
- **Feature extraction for machine learning.** The DFA verdicts are cheap, well-defined Boolean features for a downstream classifier.
- **Log and traffic analysis.** Linear-time DFAs can scan very large proxy or DNS logs in one pass with constant memory, which is the same principle behind signature engines in intrusion-detection systems.
- **Teaching.** The interface makes the regex → NFA → DFA → minimal DFA pipeline and the computation of a DFA visible on real-world input.

5.2 Limitations and Challenges

- **Keywords are evidence, not proof.** Legitimate sites use words such as *account* and *signin* (a false positive in Experiment 3).
- **Clean-looking malicious URLs pass.** URL shorteners, newly registered domains and brand imitation without keywords produce no pattern match (false negatives).
- **Evasion.** Attackers can use homoglyphs (paypa1), percent-encoding or other obfuscation that the current patterns do not cover. Lower-casing is the only normalisation performed.
- **Expressive limits of regular languages.** By the pumping lemma, DFAs cannot compare two unbounded parts of a string (for example link text against real host) or count without a bound. Such checks need more powerful models or ordinary code.
- **Project-defined scoring.** The weights and thresholds were chosen by hand and are not calibrated on data.
- **Evaluation size.** The 22-URL set is illustrative. A meaningful accuracy estimate needs a large public dataset.

5.3 Discussion

The results support the central claim of the project. Every pattern compiled from a regular expression produced a DFA that is provably equivalent to the expression, confirmed experimentally on over four million strings. Each DFA is minimal by construction and runs in time linear in the URL length. The theory explains the behaviour observed in practice. The Myhill–Nerode theorem predicts that a keyword of length k needs exactly k + 1 states, and every keyword DFA had exactly that many. The subset construction's exponential bound never materialised, because the NFAs built from these expressions have little real nondeterminism.

The project also makes the limits of the model visible. The automata answer “does this regular pattern occur?” exactly, but whether a URL is phishing is a semantic question that depends on intent, context and reputation, and it is not a regular property. Separating the precise part (regular pattern recognition) from the heuristic part (weights and thresholds) keeps the system honest: every verdict can be traced to concrete automaton runs, and every judgement call is visible in a small table of weights.

Finally, the comparison between nine separate keyword DFAs and their 44-state union shows a design trade-off allowed by the closure properties. A single product automaton is smaller and needs only one pass, while separate automata keep the individual verdicts that make the score explainable. Accepting states labelled with the keywords they recognise, as in the Aho–Corasick construction, would combine both advantages.

6. Conclusion and Future Scope

6.1 Conclusion

This project applied the models studied in Models of Computation to a real cybersecurity problem. URLs were modelled as strings over a 60-symbol alphabet, and thirteen suspicious patterns were specified as regular expressions. Each expression was compiled by the standard constructions, converted into an NFA, determinised by the subset construction and minimised by partition refinement, into a minimal DFA that scans the relevant part of the URL in linear time. The DFA verdicts and three structural rules produce a transparent risk score and class.

The implementation includes a Python engine, a Flask API and an interactive website with a live analyzer, a step-by-step DFA simulator and a construction lab. It was verified against Python's regular-expression engine on 4.07 million strings with no disagreements, and it scans URLs at about 0.389 µs per character. On an illustrative set it flagged 10 of 12 phishing-style URLs with one false positive among 10 benign URLs. The errors it made follow directly from the limits of keyword-based, regular-language recognition. The project therefore shows both the practical value of alphabets, regular expressions, NFAs, DFAs, the subset construction and minimisation, and the boundary of what these models can decide on their own.

6.2 Future Enhancements

- Add a trusted-domain allowlist and percent-decoding before the automata run, to reduce false positives and resist encoding tricks.
- Build one product DFA whose accepting states record which keywords were found (Aho–Corasick style), keeping per-keyword verdicts with a single pass.
- Add regular patterns for homoglyphs and typosquatting (for example 0 for o, rn for m, 1 for l).
- Use pushdown automata or context-free grammars for nested structures, such as URLs embedded inside query parameters.
- Integrate threat-intelligence feeds, and calibrate the weights on a large public phishing dataset.
- Compare the rule-based detector with machine-learning detectors, and use the DFA verdicts as interpretable features.

References

[1] J. E. Hopcroft, R. Motwani and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Pearson, 2006.

[2] M. Sipser, *Introduction to the Theory of Computation*, 3rd ed. Cengage Learning, 2012.

[3] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006, ch. 3 (lexical analysis, regular expressions to automata).

[4] S. C. Kleene, “Representation of events in nerve nets and finite automata,” in *Automata Studies*, Princeton University Press, 1956, pp. 3–41.

[5] M. O. Rabin and D. Scott, “Finite automata and their decision problems,” *IBM Journal of Research and Development*, vol. 3, no. 2, pp. 114–125, 1959.

[6] E. F. Moore, “Gedanken-experiments on sequential machines,” in *Automata Studies*, Princeton University Press, 1956, pp. 129–153.

[7] J. E. Hopcroft, “An n log n algorithm for minimizing states in a finite automaton,” in *Theory of Machines and Computations*, Academic Press, 1971, pp. 189–196.

[8] T. Berners-Lee, R. Fielding and L. Masinter, “Uniform Resource Identifier (URI): Generic Syntax,” RFC 3986, IETF, 2005.

[9] A. Costello, “Punycode: A Bootstring encoding of Unicode for Internationalized Domain Names in Applications (IDNA),” RFC 3492, IETF, 2003.

[10] Python Software Foundation, “re: Regular expression operations” and “urllib.parse: Parse URLs into components,” *Python 3 Documentation*.

Appendix A: Project Files and How to Run

Table 12. Project files

File	Contents
phishing_detector/automata.py	Regex parser, alphabet compression, NFA construction, subset construction, minimisation, DFA class
phishing_detector/detector.py	URL parser, pattern catalogue, structural rules, scoring and classification
app.py	Flask server and JSON API
website/index.html	Self-contained website with the JavaScript port of the engine
evaluate.py	Reproduces all experiments; writes results/results.json and results/results.md
build_site.py	Injects the results into the website
tests/test_automata.py	17 unit tests

```
pip install flask pytest
python -m pytest tests          # 17 tests
python evaluate.py              # regenerate all results
python build_site.py            # put the results into the website
python app.py                   # open http://127.0.0.1:5000
```

The website can also be opened directly from `website/index.html` without a server, because it runs the same automata in the browser.